

Real-Time Instant Messaging Application ("Pulse Messenger")

Internship Project Report - Navodita Infotech

Field	Details
Project title	Real-Time Instant Messaging Application (Pulse Messenger)
Internship domain	Web Development (MERN Stack)
Submitted by	Byreddy Rithwik Reddy
Submitted to	Navodita Infotech
Date	September 2026
Stack	MongoDB, Express.js, React, Node.js + Socket.io, Cloudinary, on-device AI translation
Repository layout	client/ (React + Vite PWA) and server/ (Express + Socket.io API)

Table of Contents

1	Executive Summary
2	Objectives
3	Scope of Work
4	Technology Stack
5	System Architecture
6	Module Breakdown
7	Feature Catalogue
8	Security Design
9	Setup and Installation Guide
10	Demo Accounts and Sample Data
11	Testing Summary
12	Challenges Faced and Solutions Applied
13	Recent Improvements (Development Log)
14	Limitations and Future Scope
15	Conclusion
16	Appendix A - API Endpoint Reference
17	Appendix B - Project Structure
18	References

1. Executive Summary

Pulse Messenger is a fully working, real-time instant messaging application built with the MERN stack and Socket.io. It supports one-to-one chats, group chats, broadcast channels, 24-hour status stories, audio/video calling (including group calls), and a wide set of modern messaging features such as quoted replies, message editing, scheduled messages, starred messages, polls, tasks, events, reminders, live location sharing, on-device translation, and end-to-end encrypted messaging.

The application is responsive across desktop and mobile, installable as a Progressive Web App (PWA). Document sharing uses a private backend store (MongoDB GridFS) with authenticated downloads, while images, audio, and video are served through Cloudinary. The project includes automated browser tests (Playwright), health-monitoring endpoints, seed data for evaluation, and full setup documentation.

2. Objectives

1. Build a real-time chat system with instant message delivery, presence (online/offline), and typing indicators.
 2. Support both private conversations and group communication with admin controls.
 3. Implement voice/video calling between users.
 4. Provide everyday messaging utilities: replies, forwarding, editing, deletion, search, media sharing, and file sharing.
 5. Apply practical security: hashed passwords, secure cookie sessions, encrypted message content, and abuse controls (block/report).
 6. Deliver a polished mobile experience: bottom-tab navigation, swipe gestures, PWA install, and an Android build path.
 7. Keep the codebase testable and documented so an evaluator can run, test, and review it end to end.
-

3. Scope of Work

- **Included:** authentication, real-time messaging (text, media, files, voice notes, locations, contacts, polls, code snippets, sketches), groups with invite codes and join approvals, channels, broadcast lists, status stories, 1-to-1 and group WebRTC calls, call history, push notifications (Web Push), chat organization (labels, pins, archive, chat lock), themes, PWA support, automated Playwright tests, seed data, and documentation.
 - **Out of scope (not attempted):** app-store publication, iOS native build, production DevOps (CI/CD, log aggregation, error-tracking service integration).
-

4. Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + Vite	Component UI, fast dev/build pipeline
State	Zustand (6 stores)	Auth, chat, calls, group calls, friends, theme
Styling	Tailwind CSS 3.4 + custom semantic theme	Responsive glassmorphism UI, dark mode
Lists	react-virtuoso	Virtualized rendering of long message histories
Real-time	Socket.io (client + server)	Messaging, presence, typing, call signalling
Backend	Node.js 18+ + Express.js	REST API, auth, file streaming
Database	MongoDB + Mongoose (13 models)	Users, messages, rooms, calls, statuses, tasks, events, reminders, labels, broadcasts, reports, tokens
File store (docs)	MongoDB GridFS (pulse_docs bucket)	Private PDF/Office/zip/txt storage with authenticated attachment downloads
Media store	Cloudinary	Images, audio, video, avatars
Auth	JWT in HttpOnly cookies + bcryptjs (12 salt rounds)	Session management, password hashing
Encryption	Web Crypto API (AES-GCM 256-bit)	Client-side end-to-end message encryption
Calls	WebRTC (peer-to-peer, Socket.io signalling, per-peer ICE queue, capped backoff)	Audio/video and group calls
Translation	Chrome Translator + LanguageDetector APIs with server gtx fallback	Free unlimited on-device translation
Push	Web Push (VAPID) + Firebase Cloud Messaging	Background message/call alerts, deep links

Layer	Technology	Purpose
PWA support	Web app manifest + service worker	Installable app Add-to-Home-Screen
Testing	Playwright (7 spec files)	Automated end-to-end browser tests
PWA	vite plugin + usePWAINstall hook	Add-to-Home-Screen install

5. System Architecture

```
+-----+ HTTPS / WSS +-----+ +-----+ | React SPA (Vite)| <--- REST + Socket.io
--> | Express API | <---> | MongoDB Atlas | | - 6 Zustand | | - 7 route groups | | - 13 models | | stores | | - JWT
cookie auth | +-----+ | - ~50 components| | - Socket.io layer | | - PWA (web only)| | - GridFS doc store|
+-----+ +-----+ +-----+ +-----+ | | Media upload/download | Media upload (signed) v v +-----+
+-----+ | Cloudinary CDN | | Push: VAPID (web) | | (images/audio/ | | rendered via | | video/avatars) | |
service worker | +-----+ +-----+
```

Key data flows

- **Send message:** client encrypts text (AES-GCM) -> POST /api/chat/... or socket emit -> server persists to MongoDB -> server emits to recipient room(s) -> recipient decrypts locally -> read/delivery receipts flow back over sockets.
- **Document share:** client uploads via POST /api/upload/document (50 MB cap, allow-listed types) -> stored in GridFS, message stores { url, fileId, name, size } -> recipient downloads via authenticated GET /api/upload/file/:id (owner-or-participant check, served as attachment).
- **Media share:** client requests /api/upload/signature -> uploads directly to Cloudinary -> URL saved on the message.
- **Calls:** WebRTC offer/answer/ICE exchanged through Socket.io rooms; call records persisted to call history; push alerts wake offline/killed apps with deep links (?chat=, ?callFrom=).
- **Push:** VAPID subscriptions stored server-side; background events trigger notifications; dead endpoints auto-prune.

6. Module Breakdown

Client (client/src/)

- pages/ - HomePage (3-zone layout: activity rail, conversation sidebar, chat workstation), LoginPage, SignUpPage, WelcomePage.
- components/ - Sidebar (search, filter tabs, swipeable mobile pager, bottom nav), ChatPane (virtualized feed, composer), MessageBubble, MessageInput (text, voice, camera, location, GIF, polls, sketches, code), plus ~45 feature modals/drawers (groups, calls, status, starred, labels, tasks, events, reminders, broadcasts, channels, themes, settings, profile).
- store/ - useAuthStore, useChatStore, useFriendStore, useCallStore, useGroupCallStore, useThemeStore.
- lib/ - axios (credentialed instance + 401 handling), crypto (E2EE), onDeviceTranslate (Chrome Translator/LanguageDetector, unlimited free translation), download (multi-stage file downloader), sound (Web-Audio synthesized tones), chatLock, backNavigation (Android back-button mapping), push, notification, vcard, chatThemes.

Server (server/src/)

- routes/ - auth, chat, upload, friends, status, calls (call.route.js), push (7 groups).
- controllers/ - 8 controllers matching the routes plus broadcast logic.
- models/ - 13 Mongoose models (User, Message, Room, Status, CallLog, FriendRequest, BroadcastList, Event, Task, Reminder, Report, DeviceToken, PushSubscription).
- lib/ - socket.js (auth + rooms + signalling), db.js (Atlas connect with pooling), corsConfig.js (origin allow-list), utils.js (token/cookie helper), messageScheduler.js (scheduled messages), push.js/notify.js (notifications), aiCompanion.js, cloudinary.js.
- middleware/ - JWT route guard (protectRoute).

7. Feature Catalogue

Authentication and accounts: signup/login/logout, JWT HttpOnly cookies, friend requests, profiles with avatar/bio/status, block/report with blocklist management.

Messaging: 1-to-1 DMs, groups (create, invite codes, join approvals, admin add/remove, leave, delete with ownership transfer), quoted replies with jump-to, editing with (edited) badges, delete-for-everyone and delete-for-me, forwarding, scheduled messages, delivery/read receipts with per-message info, typing indicators, in-chat and global message search (media/date filters), rich formatting, spoiler blur syntax, emoji reactions, quick emoji bar, jumbo emojis, Twemoji rendering.

Groups++: announcements, events with RSVP, tasks with assignees/due dates, polls with live results.

Media and files: images (compression + HD toggle, lightbox viewer, webcam capture), voice notes with waveform player, view-once photos/voice, GIFs (Giphy), stickers, link previews, vCard contact sharing/export, document sharing (PDF/Office/zip/txt via private backend store) with one-click downloads, drag-and-drop uploads, file-size formatting.

Status and social: 24-hour text/photo/video stories with viewers list, custom mood statuses, ghost/invisible mode, online presence rings.

Calls: 1-to-1 audio/video calls (mute, camera flip, speaker, screen share, PiP, peer-state badges, emoji reactions), group calls, call history with tap-to-redial, missed-call notices in chat.

Organization: Saved Messages self-chat, chat labels/folders, pin chats, archive, per-chat notification tones, mute, disappearing-message timers, whisper (view-once text) mode, chat lock behind device PIN, drafts with indicators, date dividers.

Productivity: reminders, broadcast lists (up to 50 recipients), live location sharing (15m/1h/8h), conversation export (HTML/JSON/TXT), scratchpad/quick notes with send-to-chat, code snippet composer (13+ languages), sketch/whiteboard composer, voice typing (Speech-to-Text), message readout (Text-to-Speech, on-device unlimited), on-device translation with server gtx fallback and on-device/server badges.

Mobile and desktop: responsive 3-zone layout, WhatsApp-style swipeable bottom tabs (Chats/Updates/Groups/Calls) with drag physics, safe-area support, Android back-button/gesture interception, PWA install, Capacitor Android shell with FCM, desktop notifications, Web-Audio synthesized tones, chat themes with doodle backgrounds, appearance studio with standalone dockable live preview (left/right on desktop, popup on mobile).

8. Security Design

Control	Implementation
Password storage	bcryptjs, 12 salt rounds; minimum length enforced
Sessions	JWT in HttpOnly cookies (Secure + SameSite=None in production, Lax locally); Bearer-token fallback for WebView contexts
Brute force	express-rate-limit on auth endpoints (50/15min; 1000 in test env)
Transport	CORS origin allow-list (HTTP + Socket.io; null origins rejected; *.vercel.app only with ALLOW_VERCEL_PREVIEWS=true), trust proxy, x-powered-by disabled
Payload abuse	1 MB JSON body cap; 50 MB document cap; 25 MB proxy cap; message text 8000 chars; polls 2-10 options; profile/preference length caps; allow-listed document types/MIMEs; executable/HTML/SVG uploads rejected
SSRF	isSafeUrl + manual-redirect guards on server-side fetches (blocks localhost, private ranges, metadata IPs, odd ports); preview HTML capped; proxy never reflects upstream status

Control	Implementation
Injection	Regex-escaped username lookup with collation uniqueness; Mongoose models; uniform 401 (no enumeration); ObjectId validation on room/admin/message routes; filename sanitization on downloads
Content privacy	AES-GCM 256-bit client-side E2EE (single vault, validated keys); locked chats hide previews behind validated SHA-256 device PIN; whispers/reactions/polls/threads/receipts/translation gated by participant checks
File access	GridFS downloads require owner-or-conversation-participant; served as attachment with nosniff

9. Setup and Installation Guide

Prerequisites: Node.js v18+, free MongoDB Atlas account, free Cloudinary account.

Environment variables - create server/.env:

```
PORT=5000 MONGODB_URI=your_mongodb_connection_string
JWT_SECRET=your_super_secret_jwt_key
CLOUDINARY_CLOUD_NAME=your_cloudinary_cloud_name
CLOUDINARY_API_KEY=your_cloudinary_api_key
CLOUDINARY_API_SECRET=your_cloudinary_api_secret CLIENT_URL=http://localhost:5173
VAPID_PUBLIC_KEY= VAPID_PRIVATE_KEY= VAPID_SUBJECT=mailto:you@example.com
NODE_ENV=development FIREBASE_SERVICE_ACCOUNT= ALLOW_VERCEL_PREVIEWS=false
PLAYWRIGHT=1
```

Create client/.env:

```
VITE_API_URL=http://localhost:5000 VITE_GIPHY_API_KEY=
```

Install and run:

```
# Server cd server npm install npm run seed # optional: creates demo users, a
group, and sample messages npm run dev # starts on http://localhost:5000 (use
`npm start` in production) # Client (second terminal) cd client npm install npm
run dev # starts on http://localhost:5173
```

Verify: open <http://localhost:5173>, sign up two accounts (or use the demo accounts below) in two browsers, and exchange messages. Health check: GET <http://localhost:5000/api/health> should return pong.

Install as an app (PWA): installs straight from the browser via Add to Home Screen, with background push notifications.

10. Demo Accounts and Sample Data

For evaluation without manual setup, run `npm run seed` in `server/` (an empty database also auto-seeds on first connect). This creates:

Username	Email	Password
User1	user1@example.com	password123
User2	user2@example.com	password123
User3	user3@example.com	password123
User4	user4@example.com	password123
User5	user5@example.com	password123

Seeded content: a "General Group" with welcome messages; User1 and User2 start as friends. Suggested evaluation flow: log in as User1 and User2 in two browsers -> accept/send a friend request -> exchange text, an image, and a PDF -> try a voice/video call -> post a status -> create a group.

11. Testing Summary

- **Automated (Playwright, client/tests/)**: 7 spec files covering auth, 1-to-1 chat, groups, file/media sharing, media playback, mobile navigation, and settings/profile, executed on Chromium, Mobile Chrome (Pixel 5), and Mobile Safari (iPhone 12). Final full run: **29 passed, 1 skipped, 0 failed**. Important: playwright.config.ts has its webServer block commented out, so **both dev servers must already be running** before the suite (npm start in server/ on :5000 and npm run dev in client/ on :5173, default). If port 5173 is taken by another local project, start the client on a free port (npm run dev -- --port 5174 --strictPort) and point the suite at it (PLAYWRIGHT_BASE_URL=http://localhost:5174 npx playwright test) - always verify the port serves this app (a wrong app on the port fails every test). Warm up a fresh dev server with one spec first, and prefer --workers=4 on a laptop: 8 parallel workers cold-starting Vite plus bcrypt-12 signups can exceed timeouts and flake. Final full run: **29 passed, 1 skipped, 0 failed** (Chromium, Mobile Chrome, Mobile Safari).; otherwise every test fails with page.goto: Could not connect to server. The assertion timeout is raised to 15 s (expect.timeout) because cold dev-server boot plus parallel load exceeds the 5 s default. Use plain npm start (not npm run dev watch mode) for the backend during test runs: the watcher restarts on stray file changes and drops in-flight test requests. Run from client/ with npx playwright test and inspect results via npx playwright show-report.
 - **Server smoke scripts**: npm run test:push (Web Push), test_calls_offline.mjs (call signalling), test_features.mjs (socket feature pass), plus /, /health, /api/health, /ping endpoints for uptime monitoring.
 - **Server smoke scripts**: npm run test:push (Web Push), test_calls_offline.mjs (call signalling), test_features.mjs (socket feature pass), plus /, /health, /api/health, /ping endpoints for uptime monitoring.
 - **Manual verification performed**: production vite build passes cleanly (2284 modules); node --check clean on edited server controllers/libs; oxlint 0 errors; mobile swipe pager, group creation (no auto keyboard popup), PDF download chain, on-device translation with server fallback, and dockable appearance preview exercised during development.
 - **Known test gap**: server controllers rely on per-handler checks rather than a dedicated unit-test suite; recommended as future work.
-

12. Challenges Faced and Solutions Applied

1. **Cloudinary blocks document delivery (HTTP 401)**. PDFs uploaded to Cloudinary could never be fetched back. Solution: documents moved to a private MongoDB GridFS store (pulse_docs bucket) with authenticated attachment downloads; Cloudinary kept for images/audio/video only. Files that pre-date the move and are locked in Cloudinary are detected at tap time and reported with a clear "ask the sender to re-send" message instead of a dead browser viewer page.
2. **Recipients got 403 on files after backend host changes**. The download guard matched messages by exact stored URL (protocol + host). Fix: match by stable fileId (stored on new messages) with a URL-suffix fallback for older ones.
3. **Downloads failed silently**. The downloader returned early on the first error with no user feedback. Fix: errors cascade through proxy, direct-fetch, and new-tab fallbacks, with console diagnostics at each stage.
4. **Bottom tabs opened separate pages instead of swiping**. Rebuilt as a true finger-following pager: 4 pages on one track, direction-locked drag (vertical scroll preserved via touch-action: pan-y), rubber-band edges, flick-velocity + distance snap thresholds, tappable position dots; desktop stays pinned to Chats.
5. **Keyboard popped up uninvited on group creation**. Removed autoFocus from the group-subject input so the keyboard only appears on explicit tap.

6. **New uploads could still create dead links.** The Cloudinary fallback in the document uploader was removed; if the backend store is unreachable the send now aborts with a visible error instead of storing an unfetchable link.
-

13. Recent Improvements (Development Log)

- On-device translation (free/unlimited): Chrome/Edge Translator + LanguageDetector first with on-device/server badges and per-message cache; server fallback switched to quota-free Google gtx + cache (MyMemory last; quota warnings filtered).
 - Appearance studio standalone preview: dockable left/right panel on desktop, popup on mobile, 1:1 coverage of every customizable group (backgrounds, text, bubbles, accents, presence, panels, borders, pills, senders).
 - Push reliability: corrected /push/* paths, SW-ready timeouts, explicit-endpoint unsubscribe, foreign-endpoint 409, extended FCM pruning; notification tags made unique with live permission re-read.
 - Realtime hardening: socket user-existence checks, membership-checked typing/room joins, pending-call-gated signaling, participant-checked delivery acks and message actions, scheduler atomic claims + overlap guard.
 - Calls/media: per-call timers, full track/listener cleanup, ICE backoff, audio-only fallback state, group-call ICE queues, shared audio-decode context with abort, mount-safe modals, debounced Giphy search with env key (no bundled secret).
 - Auth hardening: strict signup/login validation, uniform 401s, no token echo, no public-profile email leak, profile/prefs caps, tightened CORS and auth rate limits, no production memory-DB fallback.
 - WhatsApp-style swipeable mobile tabs (Chats / Updates / Groups / Calls) with real drag-transition physics.
 - Group-add keyboard auto-popup fix.
 - PDF/file download reliability: fileId-based authorization, removal of the Cloudinary document fallback, dead-file detection with user-facing messaging, web-only streamlined downloader.
 - Code-split secondary modals/drawers to keep the initial bundle small; lazy-loaded pages.
-

14. Limitations and Future Scope

Current limitations: demo seed accounts ship with a publicly documented password (fine for evaluation, must be disabled for any real deployment); on-device translation requires Chrome/Edge (other browsers use the server fallback; first use may download a language pack); no centralized error-tracking or request-logging service; confirmation dialogs still use native confirm/prompt in places; server-side unit test coverage is thin.

Future scope: replace native dialogs with a custom modal/toast system; add helmet security headers and structured logging; add server unit + integration tests; move scheduled-message delivery to a persistent job queue; database indexes for the file-access fallback query; admin/moderation dashboard.

15. Conclusion

The task objective - a complete, demonstrable real-time chat application - is met and exceeded: text, media, files, stories, calls, groups, and productivity tooling all work end to end across desktop and mobile from one codebase, with automated tests, seed data, and documentation supporting independent evaluation. The security fundamentals (hashed passwords, secure cookie sessions, encrypted content, rate limiting, access-checked downloads) are in place, and recent hardening rounds fixed the real defects found during testing (file delivery, download reliability, mobile navigation). The project is submitted for evaluation with setup instructions, demo accounts, and this report.

Appendix A - API Endpoint Reference

Base URL: `http://localhost:5000/api` (auth via HttpOnly JWT cookie; Bearer fallback supported).

Method	Endpoint	Purpose
POST	<code>/auth/signup</code>	Register (username, email, password)
POST	<code>/auth/login</code>	Login (email or username + password)
POST	<code>/auth/logout</code>	Clear session
GET	<code>/auth/check</code>	Current session/user
GET/PUT	<code>/users, /friends/*</code>	Sidebar users, friend requests, block/report
GET/POST	<code>/chat/rooms, /chat/rooms/join/:code</code>	Groups, invite-code join
GET/POST	<code>/chat/messages/:id, /chat/...</code>	History, send/edit/delete/star/react/forward/schedule
GET	<code>/chat/download/file?url=&filename=</code>	Authenticated CORS-safe file proxy
POST	<code>/upload/document</code>	GridFS document upload (returns fileId + url)
GET	<code>/upload/file/:id</code>	Authenticated attachment download
GET	<code>/upload/signature</code>	Signed Cloudinary media upload params
GET/POST	<code>/statuses</code>	24-hour stories (post, view, viewers, delete)
GET/POST	<code>/calls</code>	Call history, redial data, clearing
GET/POST	<code>/push/*</code>	VAPID subscriptions (correct paths; baseURL already includes /api), test push
POST	<code>/chat/message/:messageId/translate</code>	Translation (client prefers on-device; server gtx fallback + cache)
GET	<code>/, /health, /api/health, /ping</code>	Liveness/uptime checks

Real-time events (Socket.io, cookie/token authenticated): message, typing/stopTyping, messageDelivered/messageRead, room updates, call signalling (joinGroupCall, offers/answers/ICE), presence.

Appendix B - Project Structure

```
RealTime Chat Application/ |-- client/ # React 19 + Vite + Tailwind + Zustand +
PWA (web only) | |-- src/ | | |-- pages/ # HomePage, LoginPage, SignUpPage,
WelcomePage | | |-- components/ # Sidebar, ChatPane, MessageBubble/Input, ~45
modals/drawers | | |-- store/ # useAuthStore, useChatStore, useFriendStore, | |
# useCallStore, useGroupCallStore, useThemeStore | | |-- lib/ # axios, crypto
(E2EE), onDeviceTranslate, | | | # download, sound, push, | | | # chatLock,
backNavigation, vcard, themes | | |-- hooks/ # usePWAInstall | |-- tests/ # 7
```

```
Playwright spec files |-- server/ # Node + Express + Socket.io + Mongoose | |--  
src/ | | |-- routes/ # auth, chat, upload, friends, status, calls, push | | |--  
controllers/ # 8 controllers (incl. broadcast) | | |-- models/ # 13 models | |  
|-- lib/ # socket, db, corsConfig, scheduler, push/notify | | |-- middleware/ #  
JWT protectRoute | |-- seed.js # demo data seeder | |-- scripts/ # gen-vapid  
helper |-- README.md # setup guide |-- FEATURES.md # full feature inventory  
(source of truth) |-- PROJECT_REPORT.md # this report
```

References

- Repository docs: README.md (setup), FEATURES.md (feature inventory), PROJECT_REPORT.md (this report), client/tests/ (automated tests).
- Stack documentation: React, Vite, Tailwind CSS, Zustand, Socket.io, Express.js, Mongoose/MongoDB (incl. GridFSBucket), Cloudinary Upload API, Web Crypto (AES-GCM), WebRTC, Web Push (VAPID), Playwright.
- Live verification performed during development: Cloudinary delivery probes (401 on image/upload, 404 on raw/upload for legacy PDFs), production vite build, oxlint, Playwright last-run record.